

Universidad ORT Uruguay
Facultad de Ingeniería

Obligatorio 1

Entregado como requisito para la materia
Estructuras de Datos y Algoritmos 2

Juan Pagliotti — 266336
Juan Manuel Reolon — 331598

Docente: Joaquin Vigna

2026

Índice

1. Explicación de Ejercicios	3
1.1. Ejercicio 1	3
1.2. Ejercicio 2	3
1.3. Ejercicio 3	4
1.4. Ejercicio 4	4
1.5. Ejercicio 5	5
2. Justificación Temporal	6
2.1. Ejercicio 1	6
2.2. Ejercicio 2	6
2.3. Ejercicio 3	7
2.4. Ejercicio 4	7
2.5. Ejercicio 5	8
3. Declaración de Autoría	8
3.1. Ejercicio 1	8
3.2. Ejercicio 2	8
3.3. Ejercicio 3	8
3.4. Ejercicio 4	9
3.5. Ejercicio 5	9
4. Tabla Resumen	9

1. Explicación de Ejercicios

1.1. Ejercicio 1

Para realizar este ejercicio, utilizamos una lógica de hashing para transformar elementos de tipo string en valores enteros. Dado que las llaves estaban representadas por letras del alfabeto, conocíamos de antemano el conjunto posible de valores que podían tomar.

Aprovechando esto, mapeamos cada llave a una posición dentro de un array de tamaño fijo.

Para asignar cada posición del array, implementamos dos funciones que obtenían el valor ASCII de cada letra y le restaban el valor ASCII de una letra base. De esta forma, cada letra quedaba asociada a un índice único dentro del array.

Con esta lógica logramos almacenar la cantidad de llaves. Finalmente, mientras avanzamos en las salas, registrando las llaves preguntamos si tenemos esa llave necesaria para seguir avanzando, completando así la solución del problema.

1.2. Ejercicio 2

Para resolver este ejercicio, inicialmente pensamos en almacenar únicamente los puntajes acumulados en cada momento para determinar quién iba primero. Sin embargo, al analizar mejor el problema vimos que esto podía generar errores cuando aparecían valores negativos: un jugador que estuviera en primer lugar podía perder puntos y dejar de estarlo, mientras que otro pasaba al frente. Incluso esta situación podía repetirse varias veces con distintos jugadores, por lo que mantener solamente el estado actual no alcanzaba para identificar correctamente al ganador.

Por esta razón, la solución más lógica fue separar la información en dos tablas hash. La primera se utilizó para almacenar el puntaje total final de cada alumno, acumulando todos sus valores a lo largo de la entrada. Con esta tabla pudimos determinar cuál era el puntaje máximo alcanzado al finalizar el procesamiento.

Luego, realizamos una segunda pasada utilizando otra tabla hash para guardar el puntaje acumulado de cada alumno en el orden en que iban apareciendo los registros. De esta manera, fuimos reconstruyendo la evolución de los puntajes y verificando quién era el primer alumno que alcanzaba el puntaje máximo final calculado anteriormente. Para que un alumno fuera considerado ganador, debía cumplir dos condiciones simultáneamente: haber alcanzado ese puntaje durante el recorrido acumulado y que su puntaje total final coincidiera con el máximo obtenido.

Con este enfoque logramos manejar correctamente los casos con puntajes negativos y determinar de forma precisa quién fue el ganador.

1.3. Ejercicio 3

La solución modela el sistema de atención utilizando un heap binario máximo (MaxHeap), ya que esta estructura permite obtener eficientemente al paciente con mayor prioridad en cada momento.

Lo primero realizado fue tanto la estructura de la cola de prioridad como la implementación del Heap. Se lo dejó solo como Heap genérico ya que en el transcurso del obligatorio se deberá a volver a usarla pero como MinHeap, entonces simplemente se le pasa la función comparadora correspondiente.

Cada paciente se almacena en una estructura Paciente, que contiene:

- id: identificador del paciente.
- hora: hora de llegada convertida a entero.
- urgencia: nivel de urgencia.
- orden: posición en la que fue ingresado en la entrada, utilizada para desempates.

Para determinar la prioridad entre pacientes, el heap utiliza una función comparadora personalizada. Esta función implementa exactamente las reglas del problema:

- 1) Se prioriza al paciente con mayor urgencia.
- 2) Si dos pacientes tienen la misma urgencia, se prioriza al que llegó primero (menor hora).
- 3) Si también coinciden en la hora, se prioriza al que apareció primero en la entrada (menor orden de ingreso).

De esta forma, el elemento ubicado en la raíz del heap siempre representa al próximo paciente que debe ser atendido. El algoritmo funciona de la siguiente manera:

- 1) Se lee la cantidad de pacientes.
- 2) Cada paciente se inserta en el heap mediante insertar().
- 3) Una vez cargados todos los pacientes, se extraen repetidamente usando pop().

Finalmente, se imprime el identificador de cada paciente en el orden correcto de atención.

1.4. Ejercicio 4

La solución modela la galaxia utilizando un grafo ponderado no dirigido, donde:

- cada planeta representa un vértice,
- cada portal espacial representa una arista,
- y el costo de energía del portal corresponde al peso de la arista.

Como los planetas están identificados por nombres (strings), se utiliza una tabla de hash para asignar a cada planeta un índice numérico único. Esto permite trabajar eficientemente con el grafo y ejecutar el algoritmo de Dijkstra utilizando enteros en lugar de cadenas.

Durante la lectura de la entrada:

- 1) Para cada planeta leído, se verifica si ya existe en la tabla hash.
- 2) Si no existe, se le asigna un nuevo índice numérico.
- 3) Luego se agrega una arista entre ambos planetas utilizando el costo indicado.

El grafo se implementa mediante listas de adyacencia, esto debido a que por letra se indica que suele ser un grafo disperso. Como los portales son bidireccionales, cada arista se agrega en ambos sentidos. Para encontrar el camino mínimo entre el planeta origen y el destino, se utiliza el algoritmo de Dijkstra.

El algoritmo mantiene:

- un arreglo costos, donde $\text{costos}[i]$ representa la menor energía conocida para llegar al planeta i ,
- un arreglo visitados para evitar procesar un vértice más de una vez,
- y un heap mínimo que permite obtener rápidamente el vértice con menor costo acumulado.

Inicialmente:

- todos los costos se inicializan en infinito,
- excepto el origen, cuyo costo es 0.

Luego:

- 1) Se extrae del heap el vértice con menor costo.
- 2) Se recorren todos sus vecinos.
- 3) Si pasar por ese vértice mejora el costo conocido hacia un vecino, se actualiza su distancia y se inserta nuevamente en el heap.

Este proceso continúa hasta vaciar el heap. Al finalizar:

- si el costo del destino sigue siendo infinito, significa que no existe un camino y se imprime -1 ,
- en caso contrario, se imprime el costo mínimo encontrado.

1.5. Ejercicio 5

La resolución de este ejercicio se basa en modelar las incompatibilidades entre estudiantes mediante un grafo no dirigido.

- Cada estudiante representa un vértice.
- Cada incompatibilidad representa una arista entre dos vértices.

El problema consiste en determinar si es posible dividir a todos los estudiantes en exactamente dos equipos de forma que dos estudiantes incompatibles nunca queden en el mismo equipo. Esto equivale a determinar si el grafo es bipartito. Un grafo es bipartito si sus vértices pueden separarse en dos conjuntos disjuntos de manera que toda arista conecte vértices de conjuntos distintos.

Otra forma equivalente de verlo es que el grafo pueda colorearse usando únicamente dos colores, sin que dos vértices adyacentes compartan el mismo color. Para resolverlo se utilizó BFS. Y se definió un nuevo TAD de cola simple y su respectiva implementación, sencilla, tal vez sin todas las funciones, pero sí las necesarias para realizar bien el BFS. La idea del algoritmo es la siguiente:

- Se mantiene un arreglo color.
- $\text{color}[i] = -1$ significa que el vértice todavía no fue visitado.
- $\text{color}[i] = 0$ o 1 representan los dos equipos posibles.

Al comenzar un BFS desde un vértice:

- se le asigna el color 0,
- y todos sus vecinos deben recibir el color contrario,
- luego los vecinos de esos vecinos reciben nuevamente el color opuesto,
- y así sucesivamente.

Durante el recorrido pueden ocurrir dos casos:

1) El vecino no fue visitado.

Entonces se le asigna el color opuesto y se encola para seguir explorando.

2) El vecino ya tiene color.

Si posee el mismo color que el vértice actual, existe un conflicto, ya que dos estudiantes incompatibles quedarían en el mismo equipo. En ese caso el grafo no es bipartito y la respuesta es «NO». El ejercicio indica además que el grafo puede no ser conexo. Por esta razón no alcanza con ejecutar BFS desde un único vértice, ya que podrían existir otras componentes conexas sin visitar. Por eso en el main se recorren todos los vértices, y cada vez que se encuentra un vértice sin color se ejecuta un nuevo BFS desde esa componente. Si todas las componentes pueden colorearse correctamente con dos colores, entonces el grafo es bipartito y se imprime SI, en otro caso imprime NO.

2. Justificación Temporal

2.1. Ejercicio 1

La complejidad temporal de esta solución es $O(n)$, donde n representa la cantidad de salas.

Para resolver el problema se utiliza un array de tamaño fijo de 26 posiciones `cantLlaves`, correspondiente a las posibles letras del alfabeto. La inicialización de este arreglo tiene costo constante, ya que siempre se crean exactamente 26 posiciones, por lo que su complejidad es $O(1)$.

Luego, el algoritmo realiza un único recorrido sobre el string `salasLlaves`. El for avanza de dos en dos posiciones porque cada par representa una llave y la sala asociada. En cada iteración se ejecutan únicamente operaciones de tiempo constante: calcular el valor hash mediante una resta de caracteres, acceder a una posición del array, incrementar o decrementar un contador y realizar una comparación.

Como la cantidad de iteraciones es proporcional a la cantidad de salas (más precisamente `cantSalas - 1`), el tiempo total de ejecución crece linealmente respecto al tamaño de la entrada.

Por lo tanto, la complejidad temporal final del algoritmo es $O(n)$ donde n es la cantidad de salas del problema.

2.2. Ejercicio 2

La complejidad temporal de esta solución es $O(n)$, donde n representa la cantidad de registros de alumnos ingresados.

Durante la primera pasada sobre la entrada se recorren los n registros una única vez. En cada iteración se realizan operaciones sobre la tabla hash (`exists`, `get` e `insert`) que, considerando el comportamiento esperado de la tabla hash con la función de hash adecuada, tienen caso promedio $O(1)$. Por lo tanto, esta etapa completa tiene orden $O(n)$.

Posteriormente, se realiza un segundo recorrido sobre el arreglo de alumnos para determinar el puntaje máximo final. En cada iteración únicamente se consulta la tabla hash y se compara el valor obtenido, ambas operaciones de costo constante, dando nuevamente una complejidad $O(n)$.

Finalmente, se efectúa una tercera pasada para reconstruir el puntaje acumulado de cada alumno utilizando una segunda tabla hash. Al igual que antes, las operaciones `exists`, `get` e `insert` tienen en promedio $O(1)$, por lo que recorrer todos los elementos mantiene un costo total $O(n)$.

Sumando todas las etapas obtenemos: $O(n)$

Esto se logra gracias al uso de tablas de hash, que permiten realizar inserciones y consultas en orden 1 promedio, evitando búsquedas lineales sobre los alumnos ya procesados.

2.3. Ejercicio 3

La solución utiliza un heap binario máximo para mantener ordenados los pacientes según su prioridad de atención.

Cada paciente se inserta en el heap mediante la operación `insertar()`. Al insertar, el elemento puede necesitar subir posiciones dentro del heap usando `flotar()` para mantener el orden correcto. En el peor caso, puede subir desde una hoja hasta la raíz, recorriendo una cantidad de niveles proporcional a $\log N$. Por lo tanto, cada inserción cuesta $O(\log N)$.

Como se insertan N pacientes, el costo total de inserción es $O(N(\log N))$. Luego, los pacientes se extraen utilizando `pop()`. Cada extracción elimina la raíz del heap (el paciente con mayor prioridad) y reorganiza la estructura usando `hundir()`. En el peor caso, el elemento puede bajar desde la raíz hasta una hoja, recorriendo también $\log N$ niveles. Entonces, cada extracción cuesta: $O(\log N)$ y como se realizan N extracciones, el costo total es $O(N(\log N))$.

Finalmente, sumando inserciones y extracciones: $O(N(\log N)) + O(N(\log N)) = O(N(\log N))$ cumpliendo con la complejidad pedida en el ejercicio.

2.4. Ejercicio 4

La solución cumple con el orden temporal requerido $O((N + M)(\log N))$ donde:

- N es la cantidad de planetas (vértices),
- M es la cantidad de portales (aristas).

Primero, la construcción del grafo utilizando listas de adyacencia cuesta $O(M)$ ya que cada portal se procesa una vez y se agrega al grafo. Luego se ejecuta el algoritmo de Dijkstra utilizando un heap mínimo. El heap permite obtener rápidamente el vértice con menor costo acumulado. Durante la ejecución:

- cada vértice puede ser extraído del heap en tiempo $O(\log N)$
- y cada vez que se encuentra un camino más corto hacia un vecino, se inserta un nuevo elemento en el heap, operación que también cuesta $O(\log N)$

Como el algoritmo recorre todas las aristas del grafo y realiza operaciones sobre el heap para mantener ordenados los costos mínimos, el costo total resulta $O((N + M)(\log N))$ cumpliendo con la complejidad pedida en el ejercicio.

2.5. Ejercicio 5

La complejidad temporal de la solución es $O(V + A)$, donde V representa la cantidad de vértices y A la cantidad de aristas del grafo. Esto ocurre porque el algoritmo BFS recorre cada vértice una sola vez. Cuando un vértice se visita por primera vez, se le asigna un color y se agrega a la cola. A partir de ese momento no vuelve a procesarse nuevamente como no visitado, por lo que cada vértice se encola y desencola únicamente una vez. Además, durante el recorrido de cada vértice se analiza su lista de adyacencia para inspeccionar a todos sus vecinos. Utilizando listas de adyacencia, el costo total de recorrer todos los vecinos es proporcional a la cantidad total de aristas del grafo. Como el grafo es no dirigido, cada arista aparece dos veces en las listas de adyacencia: una vez para cada extremo. Sin embargo, esto sigue siendo lineal respecto a A , ya que las constantes no afectan el orden de complejidad. Por lo tanto:

- el procesamiento de vértices aporta un costo $O(V)$,
- y el recorrido de aristas aporta un costo $O(A)$.

En conjunto, la complejidad total del algoritmo es $O(V + A)$ cumpliendo con la restricción solicitada en el ejercicio.

3. Declaración de Autoría

3.1. Ejercicio 1

El ejercicio 1 fue realizado en su totalidad por los integrantes del equipo, no se utilizó IA ni se utilizó material.

3.2. Ejercicio 2

Para obtener las funciones de hash se utilizó material teórico que se encuentra en la web de la Universidad.

A su vez, también se consultó a IA, para resolver el problema de los valores negativos. Dado a que en un principio la solución que se creó no satisfacía los casos bordes que un alumno bajaba y subía del ranking.

Prompt: Como puedo mantener los puntajes de los alumnos que están por debajo del primero sin tener que utilizar variables para estos puestos. Y poder subir o bajar el ranking cuando llegan números negativos.

3.3. Ejercicio 3

Para el ejercicio 3 se utilizó como primer modelo tanto para el TAD PriorityQueue.h como para la implementación HeapImp.cpp los archivos proporcionados por el docente Joaquín Vigna en las clases teóricas disponibles en la web de la Universidad. Aunque mediante el desarrollo del ejercicio se fueron modificando. Para el resto del desarrollo del ejercicio no se utilizó IA ni se utilizó material de la Universidad

3.4. Ejercicio 4

Para el ejercicio 4 se utilizó como primer modelo tanto para el TAD Graph.h como para la implementación AdjacencyList.cpp y NodeListimp.cpp los archivos proporcionados por el docente Joaquín Vigna en las clases teóricas disponibles en la web de la Universidad.

Aunque mediante el desarrollo del ejercicio se fueron modificando. Para el resto del desarrollo del ejercicio, incluyendo las funciones auxiliares, no se utilizó IA ni se utilizó material de la Universidad.

3.5. Ejercicio 5

El ejercicio 5 fue realizado en su totalidad por los integrantes del equipo, no se utilizó IA ni se utilizó material de la Universidad.

4. Tabla Resumen

Ejercicio	Resultado
Ejercicio 1	Completo
Ejercicio 2	Completo
Ejercicio 3	Completo
Ejercicio 4	Completo
Ejercicio 5	Completo